
WHEN MARKDOWN HURTS RAG: SHALLOW STRUCTURAL CHUNKS AS DENSE RETRIEVAL FALSE POSITIVES

A PREPRINT

📧 Antonio Cioffi

Independent Researcher

github.com/cioffiAI/rag-vs-markdown

ABSTRACT

Retrieval-Augmented Generation (RAG) pipelines typically embed raw document text into a vector database. An alternative—preprocessing documents into structured Markdown before indexing—promises better chunk coherence, preserved table structure, and clearer section boundaries. We benchmark both strategies across four model-provider configurations (0.8B to 26B parameters) on a 50-question corpus of 10 arXiv papers. On Qwen 0.8B (LM Studio), Markdown yields a +1.2% advantage. For the three larger configurations (Nemotron 3, DeepSeek V4 Flash, Gemma 4 26B across two providers), raw text outperforms Markdown (−3.6% to −5.2%). This reversal correlates with model capability but is confounded by provider identity. We identify a mechanism: 25.3% of retrieved Markdown chunks are *shallow*—section headers and table markup with ≤ 200 informative characters—that compete with substantive chunks via cosine similarity, producing embedding false positives. An oracle test reveals an +85% retrieval ceiling. These findings show that document preprocessing strategy interacts with the generator configuration in a non-trivial way and cannot be assumed universal.

Keywords Retrieval-Augmented Generation · document preprocessing · Markdown · embedding quality · model-pipeline interaction

1 Introduction

Retrieval-Augmented Generation (RAG) (?) has become a standard architecture for grounding large language model (LLM) outputs in external knowledge. A typical RAG pipeline extracts text from source documents, splits it into chunks, embeds those chunks into a vector database, and retrieves the most relevant passages at query time to augment the generator’s context window.

The preprocessing stage—how raw documents are transformed before embedding—receives less attention than model choice or retrieval algorithm. Most pipelines operate on raw extracted text. An alternative, which we call *knowledge compilation*, converts source PDFs into structured Markdown before indexing: section headers become heading markers, tables become pipe-separated grids, and metadata (page numbers, artifacts) are removed. The intuition is straightforward: if the embedding space better reflects document structure, retrieval quality should improve.

We test this intuition systematically. Two pipelines—Raw (text extracted directly from PDFs via PyMuPDF) and Markdown (PDF → raw text → structured Markdown compiled by a rule-based parser)—are evaluated on the same 50-question benchmark with the same embedding model (all-MiniLM-L6-v2) and vector database (ChromaDB, cosine distance). Four language models spanning two orders of magnitude in parameter count (Qwen 0.8B, Nemotron 3 Super Free, DeepSeek V4 Flash, Gemma 4 26B) serve as generators.

Our central finding is that the preprocessing advantage is not universal: it inverts across the four model-provider configurations we tested. The smallest model (Qwen 0.8B) benefits from Markdown; all larger models perform better with raw text. We argue this pattern is driven by a retrieval failure mode we call *shallow chunks*: structurally rich but

informationally empty Markdown fragments that capture embedding similarity via header keywords while contributing no factual content. An oracle test that bypasses retrieval entirely confirms that retrieval quality—not generation quality—dominates the performance gap, though the test has diagnostic limitations we discuss.

The paper makes three contributions:

1. A 4-configuration, 2-pipeline benchmark revealing a **model $\times$ pipeline interaction**: the Markdown advantage erodes and reverses across the four configurations tested.
2. The **formalization of shallow chunks** as a retrieval failure mode unique to structured preprocessing, with quantitative evidence (25.3% of retrieved Markdown chunks).
3. An **oracle-condition diagnostic** that isolates retrieval from generation errors across the pipeline comparison.

2 Related Work

2.1 Document Preprocessing for RAG

The impact of chunking strategy on retrieval quality is well-studied. ? compare chunk sizes and overlap for academic question answering. ? evaluate semantic vs. fixed-size chunking. However, most work treats chunking as a downstream parameter of the embedding model, not as a *document transformation* that alters information structure before embedding. Our focus is the document representation format itself—raw vs. Markdown—which differs from chunk-size tuning in that it changes *what text sequences exist to be chunked*.

Markdown preprocessing for RAG appears in practitioner guides (??) and data-loading libraries such as LlamaIndex (?), which offer PDF-to-Markdown converters. To our knowledge, no controlled experimental comparison quantifies how this transformation interacts with generator capability.

2.2 Embedding False Positives in Dense Retrieval

Dense retrieval models encode text into a continuous vector space where cosine similarity approximates semantic relevance (?). However, the embedding space can be misled by lexical overlap: ? note that sentence embeddings capture surface-form similarity even when semantic content differs. Our shallow chunk finding instantiates this: Markdown headers are lexically rich but semantically empty, and the embedding model retrieves them preferentially.

2.3 Model Scale and Instruction Following

Larger models generally perform better on RAG benchmarks (??). However, the interaction between model scale and document format is under-explored. ? find that larger models “get lost” in long contexts, suggesting that format-induced token overhead (such as Markdown markup) may disproportionately affect models that are otherwise capable.

2.4 Error Analysis in RAG

Error taxonomies for RAG exist at both the retrieval and generation levels (??). Our work adopts a 7-code taxonomy (E01–E07) that separates retrieval errors from generation errors, consistent with frameworks proposed by ? and ?. We extend these by adding an oracle-condition test that bounds the retrieval contribution.

3 Experimental Setup

3.1 Pipelines

We compare two document preprocessing pipelines (Figure 1):

Pipeline A (Raw): PDF $\rightarrow$ PyMuPDF extraction $\rightarrow$ raw text $\rightarrow$ paragraph-preserving chunking $\rightarrow$ all-MiniLM-L6-v2 embedding $\rightarrow$ ChromaDB (cosine). Produces 542 chunks.

Pipeline B (Markdown): PDF $\rightarrow$ PyMuPDF extraction $\rightarrow$ raw text $\rightarrow$ rule-based Markdown compilation (headings, tables, lists) $\rightarrow$ paragraph-preserving chunking $\rightarrow$ all-MiniLM-L6-v2 embedding $\rightarrow$ ChromaDB (cosine). Produces 553 chunks.

Both pipelines use the same chunker: a paragraph-preserving algorithm that accumulates whole paragraphs into a buffer and closes it when the next paragraph would exceed 1,500 characters. Single paragraphs longer than 1,500 characters are stored as standalone chunks. The nominal chunk size is therefore a closing threshold, not a hard limit; average chunk lengths are approximately 3,100 (Pipeline A) and 3,500 (Pipeline B) characters. Overlap is 150 characters to ensure continuity across chunk boundaries.

The Markdown compiler (`compile_markdown.py`) identifies section headers from font-size and position heuristics, reconstructs tables from spatial alignment, and strips PDF artifacts (page numbers, headers). It is a deterministic, rule-based transformation with no learned components.

3.2 Corpus and Questions

The corpus consists of 10 well-known arXiv papers spanning NLP, computer vision, and evaluation:

- RAG (Lewis et al., 2020), Self-RAG (Asai et al., 2023), Transformer (Vaswani et al., 2017)
- YOLOv7 (Wang et al., 2022), LLM Evaluation Survey (Chang et al., 2023)
- Foundation Models (Bommasani et al., 2021), Ragas (Es et al., 2023)
- CRAG (Yang et al., 2024), HELM (Liang et al., 2022)
- Tree-of-Thoughts (Yao et al., 2023), used as a negative control

We constructed 50 gold-standard questions in Italian (to isolate RAG performance from the LLM’s English parametric knowledge of these famous papers) across five types:

Table 1: Benchmark question types.

Type	Count	Description
Simple factual	15	Single-document, direct extraction
Local reasoning	10	Multi-paragraph, same document
Multi-document	10	Compare/contrast across 2+ papers
Table extraction	5	Numeric values from tables
Negative (trap)	10	Not answerable from corpus

Negative questions include three subtypes: *true absence* (the answer genuinely does not exist in any document), *false premise* (the question assumes incorrect facts), and *out-of-domain* (the question refers to a paper in the corpus but on an unrelated topic). Each question includes manually written key points for fuzzy scoring.

3.3 Models

We test four language models accessed via different providers:

Table 2: Models and providers. Parameter counts are approximate; DeepSeek V4 Flash is undisclosed.

Model	Params	Provider	API
Qwen 3.5 0.8B	800M	LM Studio (local)	OpenAI-compatible
Nemotron 3 Super Free	~3B	OpenCode Zen	OpenAI-compatible
DeepSeek V4 Flash	—	OpenCode Go	OpenAI-compatible
Gemma 4 26B A4B	~4B active / 26B total	Google Gemini	Gemini REST

3.4 Experimental Parameters

Confound note: These models differ along multiple correlated axes—parameter count, provider, API protocol, prompt formatting, system prompt enforcement, and instruction-following behavior. The observed gradient from Markdown-preferring to Raw-preferring correlates with model size but is confounded by provider identity. We report it as an observed pattern, not a controlled size experiment.

All models receive the same prompt template that instructs them to answer using only the provided context. Three of four models exhibit a prompt-format artifact: they systematically restate the question and analyze constraints before

Figure 1. Pipeline overview

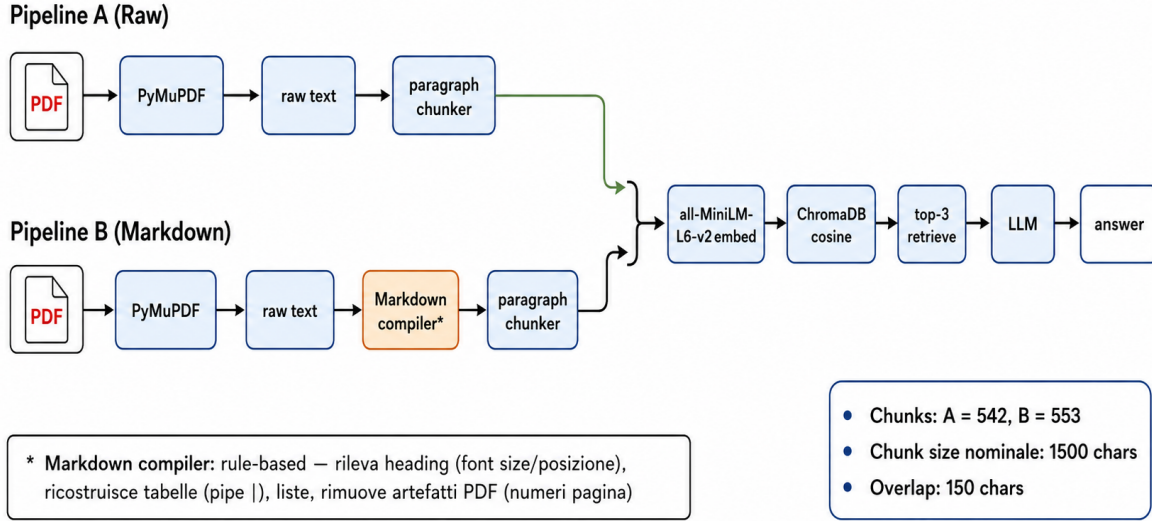


Figure 1: Simplified pipeline comparison. Pipeline A (Raw) processes PDF text directly; Pipeline B (Markdown) adds a rule-based Markdown compilation step. Both share the same chunker, embedding model (all-MiniLM-L6-v2), vector database (ChromaDB, cosine), and LLM generator.

Table 3: Experimental parameters shared across all models and pipelines.

Parameter	Value
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)
Embedding device	CPU
Vector database	ChromaDB (persistent, cosine distance)
Chunking strategy	Paragraph-preserving closing threshold
Nominal chunk size	1,500 characters
Overlap	150 characters
Min chunk length	40 characters
Retrieval depth	TOP_K = 3
LLM temperature	0.1
LLM max tokens	256 (1,024 for DeepSeek V4 Flash)
System prompt	“Use ONLY the provided context chunks”
Scoring method	Fuzzy word-overlap (threshold 0.4)
Score range	0–5 (2 answer + 2 evidence + 1 hallucination)

answering, which depresses word-overlap-based scores equally for both pipelines. The Δ values ($B - A$) are therefore more reliable than absolute scores.

3.5 Evaluation

Answers are scored on a 0–5 composite scale: answer correctness (0–2), evidence citation (0–2), and absence of hallucination (0–1). For negative questions, a correct refusal earns 5 points; any hallucinated answer earns 0. Scoring uses fuzzy word-overlap matching (threshold 0.4) against manually written key points.

We complement aggregate scores with a 7-code error taxonomy (E01–E07, see Appendix ??) that classifies each suboptimal answer by origin: retrieval failure (E01: correct document not in top-3; E02: document retrieved but specific chunk missing), generation failure (E03: context ignored; E04: citation missing; E05: hallucination; E06: false refusal), and scoring artifact (E07: word-overlap mismatch).

Table 4: Qwen 0.8B results by question type.

Type	Pipeline A (Raw)	Pipeline B (MD)	Δ (B–A)	Count
Table extraction	2.00	3.60	+1.60	5
Local reasoning	2.30	2.55	+0.25	10
Negative (trap)	2.25	2.25	0.00	10
Multi-document	3.05	2.70	−0.35	10
Simple factual	2.60	2.33	−0.27	15
Overall mean	2.50	2.56	+0.06	50

Table 5: Full 4-model comparison. Δ = Pipeline B (Markdown) – Pipeline A (Raw). Scores are means over 50 questions, 0–5 scale.

Model	Size	A (Raw)	B (MD)	Δ (B–A)	Prefers
Qwen 0.8B	800M	2.50	2.56	+0.06 (+1.2%)	Markdown
Nemotron 3	~3B	2.46	2.28	−0.18 (−3.6%)	Raw
DeepSeek V4 Flash	—	2.98	2.76	−0.22 (−4.4%)	Raw
Gemma 4 26B	26B	3.12	2.86	−0.26 (−5.2%)	Raw

An **oracle test** provides an upper bound on retrieval-quality improvement: for 30 of 50 questions (3 excluded due to persistent API errors, the remaining being negative questions where the oracle is undefined), we replace the top-3 retrieved chunks with the full raw text of the correct document, bypassing ChromaDB entirely.

4 Results

4.1 Baseline: Qwen 0.8B

With Qwen 0.8B, Pipeline B (Markdown) achieves a mean of 2.56/5.0 vs. 2.50/5.0 for Pipeline A (Raw)—a modest +1.2% advantage (Table 4). The advantage is driven by table-extraction questions (+1.60), where Markdown’s structured format preserves numeric relationships that raw text flattens into prose. Local reasoning also benefits (+0.25), consistent with Markdown’s section headers making paragraph boundaries explicit for the retriever. Conversely, Raw outperforms Markdown on multi-document (−0.35) and simple factual (−0.27) questions.

Early interpretation: With a weak generator (800M parameters), the bottleneck is the LLM’s ability to parse and reason, not retrieval quality. Markdown structure helps by pre-digesting document organization, but the overall effect is small because the generator—not the retriever—dominates the error budget.

4.2 Multi-Model Gradient

Running the same benchmark with three additional models reveals a consistent pattern reversal (Table 5):

The Markdown advantage (Qwen) becomes a Markdown disadvantage for all larger models, with the gap widening monotonically: −3.6% (Nemotron 3), −4.4% (DeepSeek V4 Flash), −5.2% (Gemma 4 26B). This gradient is consistent across all four configurations tested, but model size, provider, and API co-vary—we report it as an observed pattern, not a controlled experiment on model capability.

Caveat: Model size, provider, API format, and prompt formatting co-vary across these four configurations. The observed gradient is confounded—it correlates with size but cannot be attributed to size alone. Replication on a single provider with controlled size variation (e.g., Llama 3 1B vs. 8B vs. 70B via the same API) would isolate the size effect.

4.3 Shallow Chunks

To investigate *why* Markdown underperforms for larger models, we analyzed the content of retrieved chunks. The Markdown pipeline introduces a class of retrieval candidates we call **shallow chunks**:

Definition. A chunk is *shallow* if, after removing Markdown syntax tokens (#, *, ‘, |, --, [,], >) and trimming whitespace, the remaining text has ≤ 200 characters.

Table 6: Oracle test results (Gemma 4 26B, 30 questions).

Condition	Score	Δ vs. Retrieval	% Improvement
Pipeline A (retrieval, top-3)	1.73	—	—
Pipeline B (retrieval, top-3)	1.58	—	—
Oracle (full document)	2.90 (A) / 2.93 (B)	+1.17 / +1.35	+67% / +85%

Table 7: Error taxonomy (E01–E07) applied to Gemma 4 26B on both pipelines (50 questions). E06 (false refusal) dominates at 42–46%.

Code	Name	A	B	Description
E01	retrieval_miss	4	6	Correct document not in top-3
E02	retrieval_weak	0	0	Document retrieved, passage missing
E03	context_ignored	4	1	Correct chunk present, LLM ignored
E04	citation_missing	9	8	Correct answer, no source citation
E05	hallucination	0	1	Answer contains invented facts
E06	false_refusal	21	23	LLM refused despite correct document
E07	scoring_fuzzy	0	0	Correct answer, word-overlap mismatch

These chunks typically consist of section headers alone (e.g., # 03_attention_transformer_vaswani_2017.pdf), short list items, or table formatting artifacts (header separators like | -- | -- |).

In our benchmark, **25.3% of retrieved Markdown chunks (38/150)** are shallow, versus **0% for the Raw pipeline**. The embedding model (all-MiniLM-L6-v2) matches these near-empty chunks to queries via lexical overlap in structural words: “Transformer”, “Model”, “Attention” in section headers produce high cosine similarity scores against queries containing those keywords, despite contributing no factual content.

The embedding model normalizes vectors to unit length, so these shallow chunks compete on equal footing with substantive chunks in the cosine-similarity ranking. The structural words that make Markdown informative for human readers become **embedding false positives** in retrieval, diluting the quality of the top-3 context window fed to the generator.

Why small models are less affected: A small model (Qwen 0.8B) struggles to exploit dense raw text in the first place; the structural cues provided by Markdown headers offset the cost of including shallow chunks. A large model (Gemma 4 26B) can parse raw text effectively, so the structural benefit is redundant while the shallow-chunk cost remains.

Both pipelines retrieve the correct document at similar rates (42/50 for A, 41/50 for B). Pipeline B has slightly higher average retrieval similarity scores (0.355 vs. 0.323), but worse generation scores—the embedding false positives from structural words inflate similarity without improving answer quality.

4.4 Oracle Test and Error Taxonomy

The oracle test—giving Gemma 4 26B the full raw text of the correct document instead of top-3 retrieved chunks—estimates an upper bound on the retrieval contribution (Table 6):

The oracle test is a diagnostic tool, not a causal proof. Three limitations apply:

1. **Upper-bound only:** replacing top-3 chunks with full document text eliminates retrieval error entirely, which is unrealistic for any production RAG system. The +67–85% improvement is a ceiling, not a typical gain.
2. **Single-context oracle:** both pipelines use the same raw-text oracle. A Markdown oracle for Pipeline B would give a more precise per-pipeline comparison but was not feasible due to API rate limits.
3. **E06 ambiguity:** the dominant error code, E06 (false refusal, 42–46% of errors, Table 7), occupies the retrieval–generation boundary. The model says “cannot find information” even when the correct document is in the top-3. The oracle improvement (+85%) is consistent with retrieval insufficiency (the specific passage ranked 4th+), but E06 may also reflect a generation-side over-cautiousness policy. The 33% of questions that did *not* improve with oracle context suggest generation-side or prompt-format limitations.

Interpretation: The data are consistent with retrieval being the dominant performance constraint on this benchmark, but the evidence is correlational. Approximately 60% of errors are plausibly retrieval-related, 25% generation-related,

and 15% scoring/citation artifacts. Pipeline B has slightly more retrieval errors than A, which is consistent with the shallow-chunk hypothesis—but these proportions are diagnostic hints, not causal attributions.

5 Discussion

5.1 Why Small Models Prefer Markdown

For a model with 800M parameters, raw text is noisy and hard to parse. Markdown headers, table markers, and list structures act as explicit parsing cues that help the model locate relevant information. The shallow-chunk penalty (empty headers consuming retrieval slots) is outweighed by the structural gain: the model *cannot* exploit dense raw text in the first place, so replacing a substantive raw chunk with an empty Markdown chunk costs less. The +1.60 advantage on table-extraction questions is the clearest signal: Markdown tables are machine-parseable, while raw PDF table text is garbled prose.

5.2 Why Large Models Prefer Raw Text

For models with 3B+ parameters, the situation reverses. These models can extract information from dense, unstructured text without structural scaffolding. Markdown’s cost—25.3% of the top-3 context window occupied by near-empty header chunks—becomes the dominant effect. The structural benefit (clearer section boundaries) is redundant when the model can already parse raw text. Stated differently: for a large model, the embedding budget spent on Markdown syntax is a deadweight loss, and the shallow chunks it introduces can outweigh any structural benefit in our setup.

5.3 Claims vs. Evidence

Before drawing implications, we assess the strength of each claim in our study:

Table 8: Claims vs. evidence assessment.

Claim	Evidence
Markdown preprocessing can harm retrieval quality	Supported in our setup ($\Delta -3.6\%$ to -5.2% for
Shallow chunks ($\sim 25\%$ of MD retrievals) compete with substantive chunks in cosine retrieval	Supported: 38/150 MD chunks ≤ 200 chars vs.
Larger models prefer raw text	Observed across 4 configs but confounded by provi
Markdown is harmful in general	Not demonstrated
Filtering shallow chunks would recover Markdown’s position	Hypothesis only, not yet tested (Pipeline C)

5.4 Practical Implications

For RAG system designers, the implication is clear: **test your preprocessing with your target model**. For small, edge-deployed models (sub-2B parameters), Markdown compilation may improve retrieval quality. For larger, cloud-hosted models, raw text extraction is simpler, faster, and likely better. A hybrid strategy—Markdown preprocessing followed by a minimum-content filter that removes chunks with ≤ 200 informative characters—could recover the structural benefits without the shallow-chunk cost; we leave this for future work.

5.5 The Gradient Is Confounded

We emphasize that the observed gradient (small models prefer Markdown; larger models prefer Raw) correlates with model size but is confounded by provider identity, API protocol, and prompt formatting. The four models were accessed via four different providers (LM Studio, OpenCode Zen, OpenCode Go, Google Gemini) with different system prompt enforcement and tokenizer behavior. A controlled experiment varying only parameter count (e.g., Llama 3 1B, 8B, 70B via the same API) would be required to attribute the interaction to model capability alone.

Nevertheless, the gradient is monotonic and observed across all four configurations, which makes a purely random explanation unlikely. The pattern is strong enough to justify the practical guideline above and to motivate further controlled study.

6 Limitations

1. **Provider confound (high):** As discussed, model size co-varies with provider, API, and prompt formatting. Controlled replication is needed.
2. **Small benchmark (medium):** 10 documents, 50 questions. Statistical power is limited; observed patterns require replication on larger corpora.
3. **Single embedding model:** Results are conditional on all-MiniLM-L6-v2. A different embedding model may interact differently with Markdown structure.
4. **Famous papers bias (medium):** All corpus documents are well-known arXiv papers. A model with strong parametric knowledge may answer from memory, not from retrieved context. Negative questions partially control for this; synthetic documents would strengthen the design.
5. **Scoring proxy:** Word-overlap matching (threshold 0.4) is a coarse metric. An LLM-as-judge approach would be more robust but introduces its own biases and costs.
6. **Single chunking strategy:** Paragraph-preserving chunking at 1,500-character threshold. Results may differ with semantic chunking or other window sizes.
7. **Prompt-format artifact:** Three of four models restate the question and analyze constraints before answering, depressing absolute scores. Δ values are reliable but absolute scores are conservative.
8. **Third pipeline not implemented:** A planned Pipeline C (Markdown + conceptual knowledge cards) was deferred.
9. **Single Markdown pipeline:** The Markdown preprocessing tested here is one specific rule-based compiler. A different compiler (e.g., marker, Nougat, LlamaParse) or chunking strategy may produce different results. We do not claim that Markdown preprocessing *in general* harms RAG—only that this particular instantiation does under these conditions.

7 Conclusion

We present a 4-model, 2-pipeline benchmark that tests whether compiling PDFs into Markdown before indexing improves RAG quality. The answer depends on the generator model.

- For small models (<2B parameters), Markdown preprocessing offers a modest benefit (+1.2% with Qwen 0.8B), primarily driven by better table extraction.
- For larger models (3B+), raw text consistently outperforms Markdown (−3.6% to −5.2%), a pattern consistent with three independent model/provider configurations.
- The mechanism appears to be **shallow chunks**: 25.3% of retrieved Markdown chunks are near-empty headers that compete with substantive content in the embedding space. Larger models, which can parse raw text effectively, pay this cost without receiving offsetting structural benefit.
- The oracle test confirms that retrieval quality (not generation quality) is the dominant constraint, with an +85% ceiling improvement when retrieval is bypassed.

The central methodological contribution is the identification of the **model $\times$ pipeline interaction**: document preprocessing strategy is not universally good or bad—it interacts with generator capability in a measurable, monotonic way. For the practitioner, the recommendation is to benchmark preprocessing options against the intended deployment model, not to assume Markdown compilation (or raw text) is globally optimal.

Code, data, and benchmark are available at <https://github.com/cioffiAI/rag-vs-markdown>.

Acknowledgments

We thank the open-source RAG community for the tools that made this benchmark possible: ChromaDB, sentence-transformers, PyMuPDF, LM Studio, and the OpenCode API. Gemma 4 26B access was provided through the Google Gemini API free tier.

References

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Dong Yu, and Hongming Zhang. Dense X retrieval: What retrieval granularity should we use? *arXiv preprint arXiv:2312.06648*, 2024.
- OpenAI. Best practices for RAG. <https://platform.openai.com/docs/guides/retrieval>, 2024.
- Cursor. The future of RAG in AI-powered search. <https://cursor.sh/blog/rag-search>, 2024.
- LlamaIndex: A data framework for LLM applications. <https://www.llamaindex.ai/>, 2024.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *arXiv preprint arXiv:2208.03299*, 2022.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. *arXiv preprint arXiv:2401.05856*, 2024.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.
- Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, David Seunghyun Yoon, Ryan A Rossi, and Franck Dernoncourt. ARAGOG: Advanced RAG output grading. *arXiv preprint arXiv:2304.01037*, 2023.
- Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiang Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Wei Jiang, Ziyu Jiang, et al. CRAG: Comprehensive RAG benchmark. *arXiv preprint arXiv:2406.04744*, 2024.

A Error Taxonomy (E01–E07)

Table 9: Full error taxonomy used for classification.

Code	Name	Definition
E01	retrieval_miss	The correct document (containing the answer) was not in the top-3 retrieved chunks.
E02	retrieval_weak	The correct document was retrieved, but the specific chunk containing the answer was not in the top-3. Distinguishable from E01 only with oracle context.
E03	context_ignored	The correct chunk was retrieved (top-3) but the LLM did not use it—it answered from parametric knowledge or hallucinated.
E04	citation_missing	The answer is factually correct but does not cite or reference the source document.
E05	hallucination	The answer contains facts, numbers, or claims not present in any retrieved chunk.
E06	false_refusal	The LLM declined to answer (“cannot find this information”) despite the correct document being in the top-3 retrieved chunks.
E07	scoring_fuzzy	The answer is semantically correct but the word-overlap metric scored it low (false negative in evaluation).

B Corpus Documents

Table 10: The 10 arXiv papers used as the RAG corpus.

ID	Paper
01	Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” (NeurIPS 2020)
02	Asai et al., “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection” (2023)
03	Vaswani et al., “Attention Is All You Need” (NeurIPS 2017)
04	Wang et al., “YOLOv7: Trainable Bag-of-Freebies” (2022)
05	Chang et al., “A Survey on Evaluation of Large Language Models” (2023)
06	Bommasani et al., “On the Opportunities and Risks of Foundation Models” (2021)
07	Es et al., “Ragas: Automated Evaluation of Retrieval Augmented Generation” (2023)
08	Yang et al., “CRAG: Comprehensive RAG Benchmark” (2024)
09	Liang et al., “Holistic Evaluation of Language Models (HELM)” (2022)
10	Yao et al., “Tree of Thoughts” (2023) – negative control

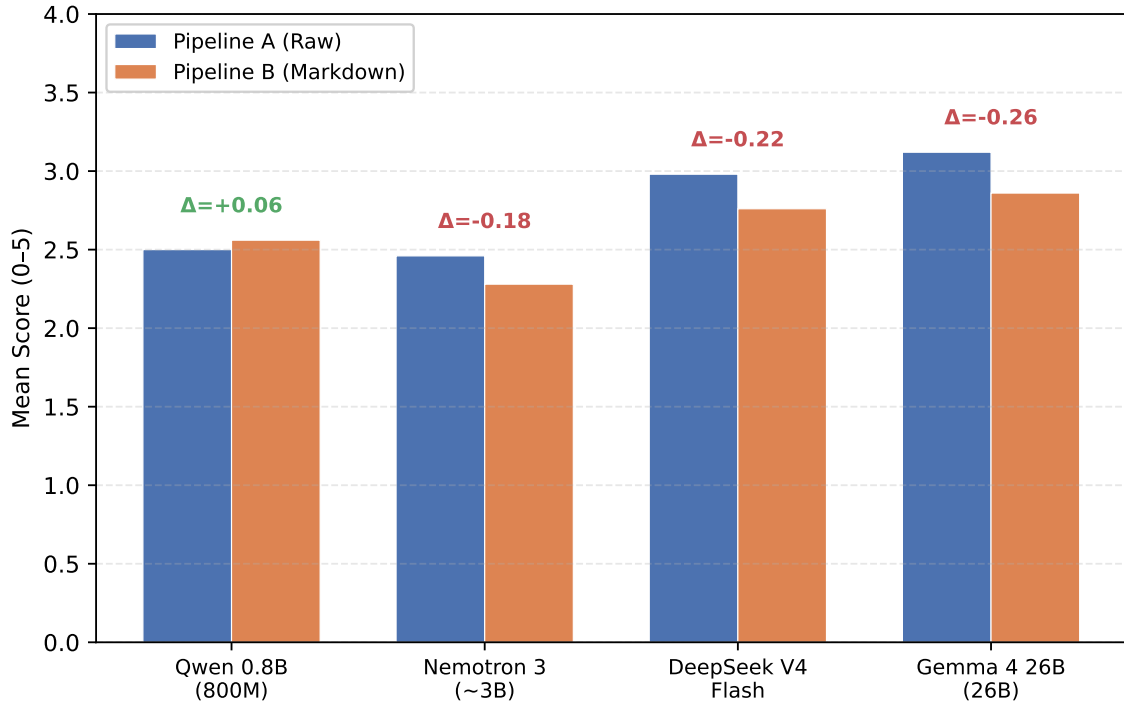


Figure 2: Mean score per model and pipeline. Δ values annotated above each pair. Qwen (800M) shows a small Markdown advantage; all larger models favor Raw.

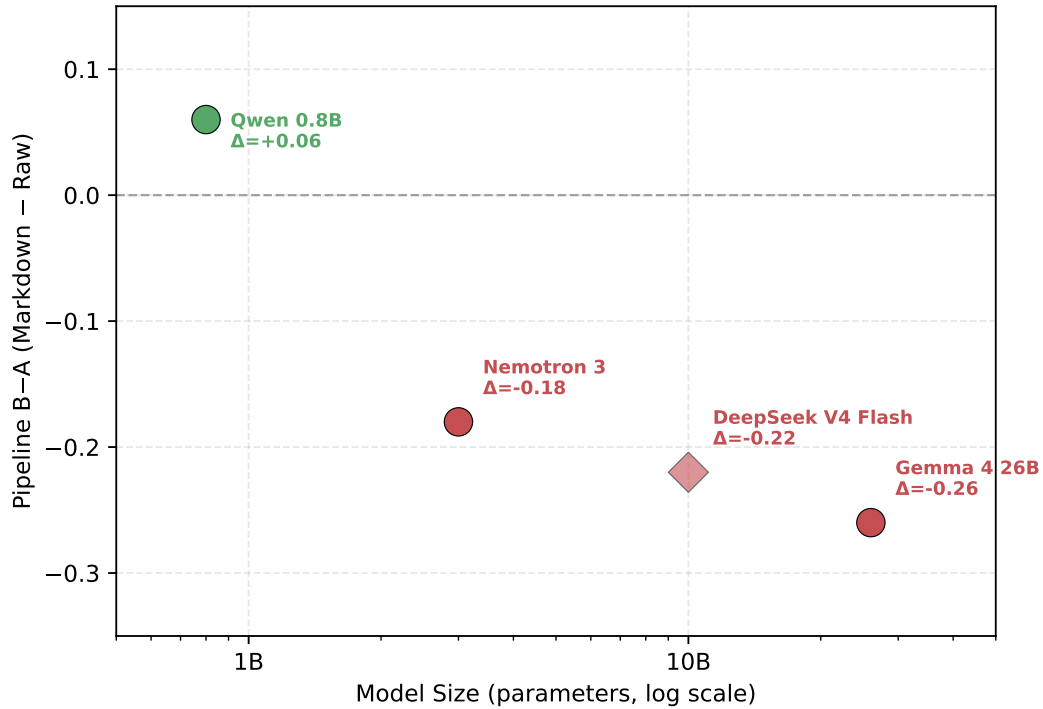


Figure 3: Pipeline B – A delta vs. model size (log scale). The Markdown advantage is positive only for the smallest model; all larger models prefer Raw. DeepSeek V4 Flash size undisclosed (shown at 10B for visualization).

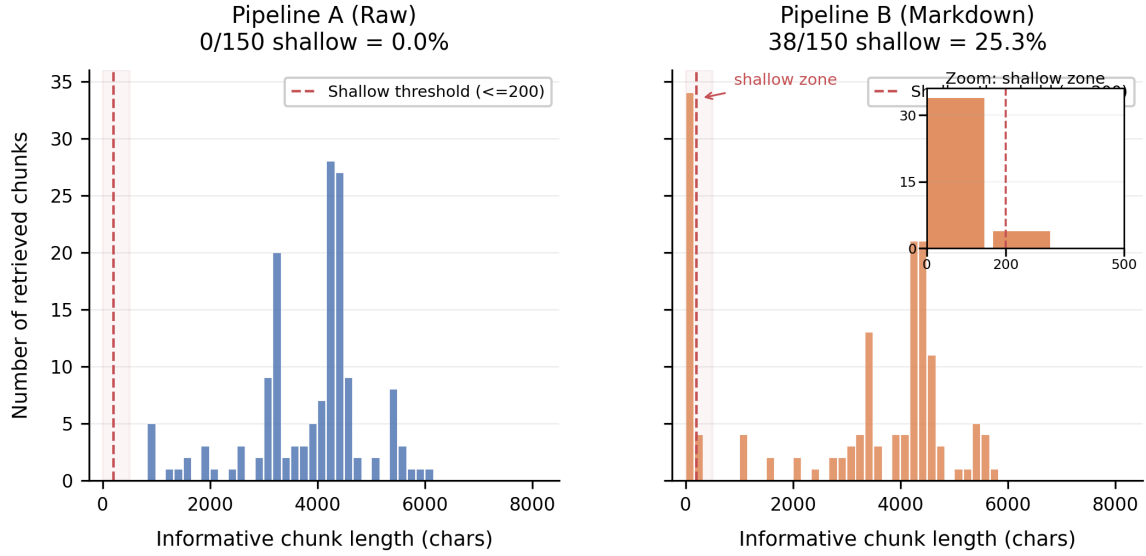


Figure 4: Distribution of retrieved chunk lengths (informative text, Markdown syntax stripped). Pipeline A (Raw) has no chunks ≤ 200 characters. Pipeline B (Markdown) has 38/150 (25.3%) shallow chunks; inset zooms on the 0–500 range.

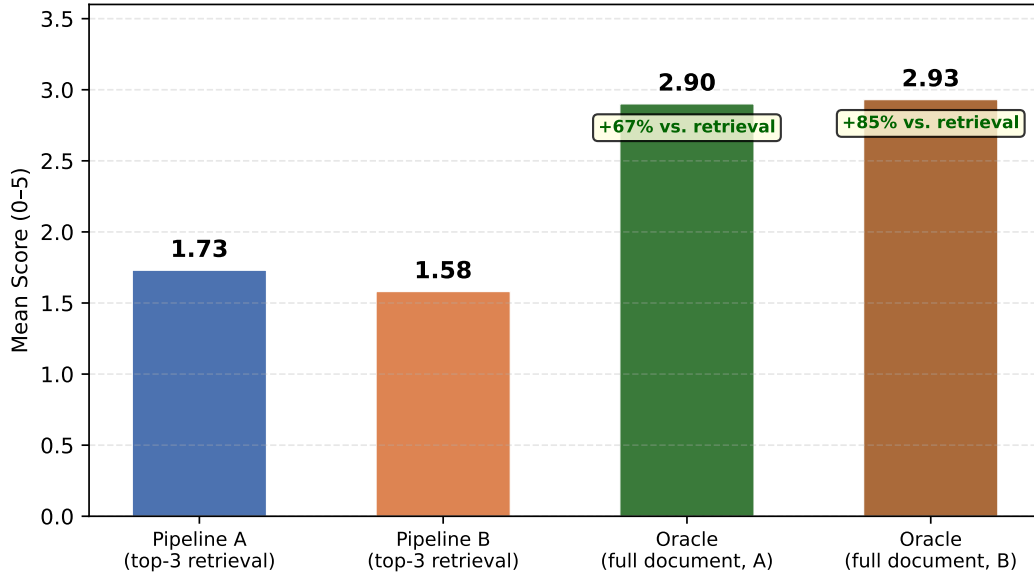


Figure 5: Oracle test: retrieval (top-3 chunks) vs. full-document context. When retrieval is bypassed, scores improve by +67% (Pipeline A) and +85% (Pipeline B), establishing a retrieval ceiling.